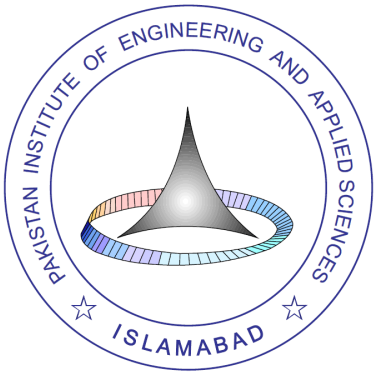
Attendance Management System

Course: Data Structures and Algorithms (DSA)



Submission Date: 15/12/2024

Submitted To: Sir Naveed Akhtar

Submitted By:

- Iqra Iqbal
- Iqra Batool
- Fatima Zahra

Department Of Computer and Information Sciences
Pakistan Institute of Engineering and Applied Sciences
P. O. Nilore, Islamabad, Pakistan

1. Introduction

1.1 Overview

The Attendance Management System is a data-driven project designed to manage and track student attendance efficiently. The project leverages **linked lists** as the primary data structure for managing records and incorporates file handling for data persistence. The system ensures data integrity through validation mechanisms and aims to streamline attendance tracking for educational institutions.

1.2 Objective

- To implement an attendance system using **DSA concepts**, particularly linked lists.
- To validate and manage student data with efficiency and accuracy.
- To demonstrate how file handling ensures persistent storage of student data.

1.3 Scope

- Add, update, search, display, and delete student records.
- Prevent duplicate roll numbers.
- Save and load student data via CSV file handling.

---

2. System Design

2.1 System Architecture

- The project integrates two main components:
    1. **Data Structures Module**: Implements linked lists for dynamic storage and record manipulation.
    2. **File Handling Module**: Manages permanent storage and retrieval of records.

2.2 Modules and Their Functions

1. **Student Class**:
    - **Attributes**: `rollNumber`, `name`, `attendancePercentage`.
    - **Methods**: Constructor for initialization, getters, and setters.
2. **LinkedList Class**:
    - Stores student records dynamically.
    - Provides methods for insertion, deletion, search, and display operations.
3. **File Management Module**:
    - Saves student records to a CSV file upon program termination.
    - Loads data from the file at the start of the program.

2.3 Data Flow

- **Input**: Student data including roll number, name, and attendance percentage from the user.
- **Processing**: Validates and manipulates data using linked list operations.
- **Output**: Displays student records or relevant error messages.

---

3. Algorithmic Approach

3.1 Linked List Operations

1. **Add Student**
   - Traverse the list to check if the roll number exists.
   - Append a new node with student details if the roll number is unique.
2. **Delete Student**
   - Locate the node based on roll number and update pointers to remove the node.
3. **Search Student**
   - Traverse the list to find the node with the matching roll number.
4. **Update Attendance**
   - Modify the attendance percentage for the matching roll number.
5. **Display Records**
   - Traverse the entire list and print each node's data.

3.2 Validation Logic

- **Roll Number Uniqueness**: Checked during the `addStudent()` operation.
- **Name Validation**: Ensures only alphabetic characters and spaces are accepted.
- **Attendance Range Validation**: Confirms attendance is within the range of 0–100%.

---

4. Implementation Details

4.1 Classes

4.1.1 Student Class

Stores attributes like roll number, name, and attendance percentage, along with getter and setter methods for encapsulation.

4.1.2 LinkedList Class

- **Node Structure**: Contains a `Student` object and a pointer to the next node.
- **Methods**:
  - `addStudent()`: Adds a student after validation.
  - `deleteStudent()`: Removes a node by matching roll number.
  - `rollNumberExists()`: Prevents duplicate roll numbers.
  - `displayStudents()`: Prints all student records sequentially.

4.1.3 File Handling Module

- **Saving Data**:
  - Writes student details (roll number, name, attendance) to a CSV file during program exit.
- **Loading Data**:
  - Reads records from the file during program initialization and populates the linked list.

4.2 Menu Options

1. **Add Student**: Validates and appends new records.
2. **Delete Student**: Removes records by roll number.
3. **Search Student**: Finds and displays details of a specific student.
4. **Update Attendance**: Edits attendance for an existing student.
5. **Display Records**: Shows all student details in the list.

---

5. Code Flow Explanation

1. **Startup**:
   - The program initializes the linked list by reading data from the file (if it exists).
2. **User Interaction**:
   - Displays a menu of options for record management.
   - Takes user input to perform desired operations.
3. **Data Handling**:
   - Uses linked list methods to manipulate in-memory records.
   - Writes updated records to a file upon exit.
4. **Shutdown**:
   - Saves all data to the file before terminating.

---

6. Performance Analysis

6.1 Efficiency

- Insertion and deletion: **O(n)** (linear traversal of linked list).
- Search operation: **O(n)** (traverse list until a match is found).

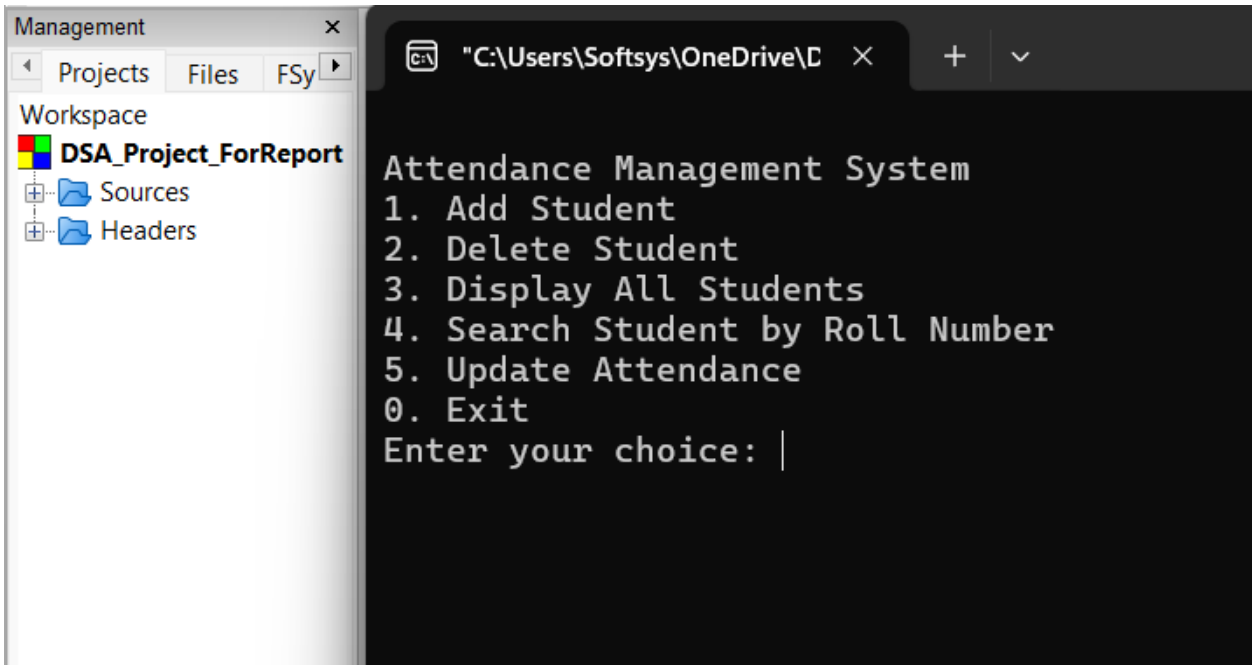
6.2 Scalability

- The system is capable of handling moderately large datasets due to the dynamic nature of linked lists.
- File handling ensures that records are retained across sessions.

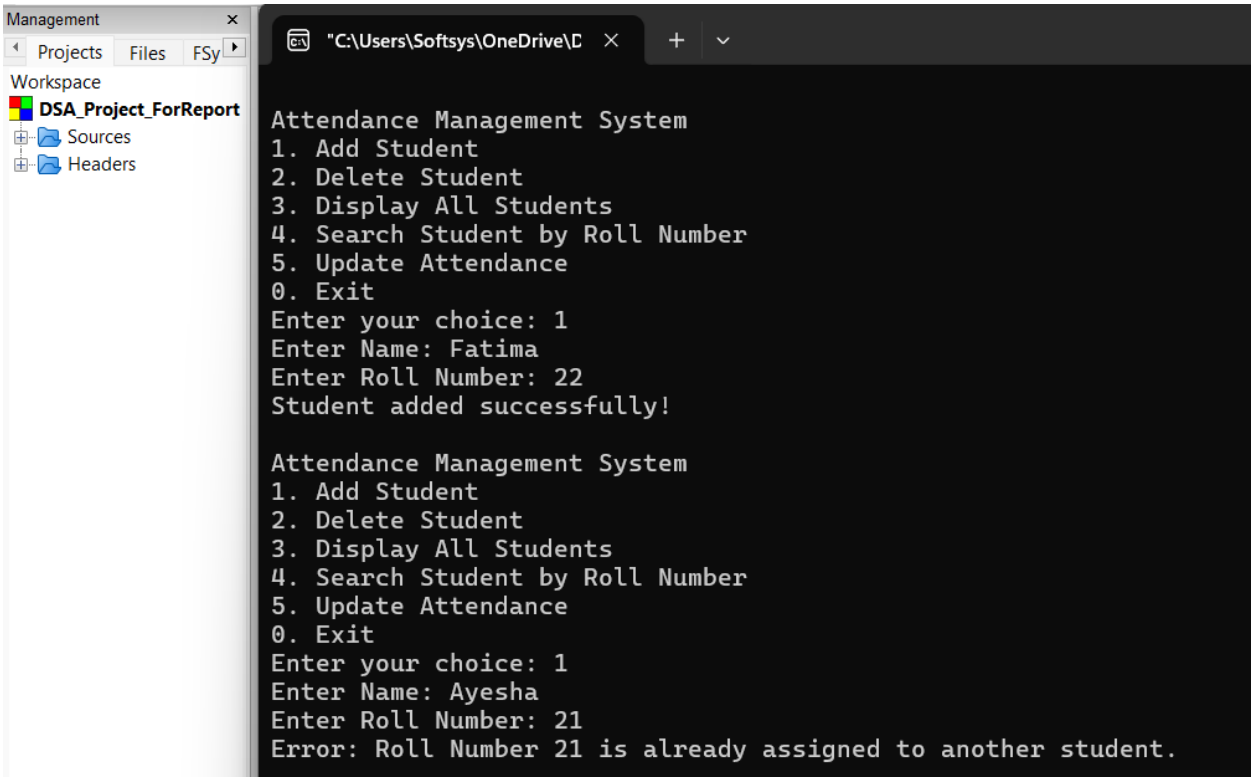
---

7. Screenshots

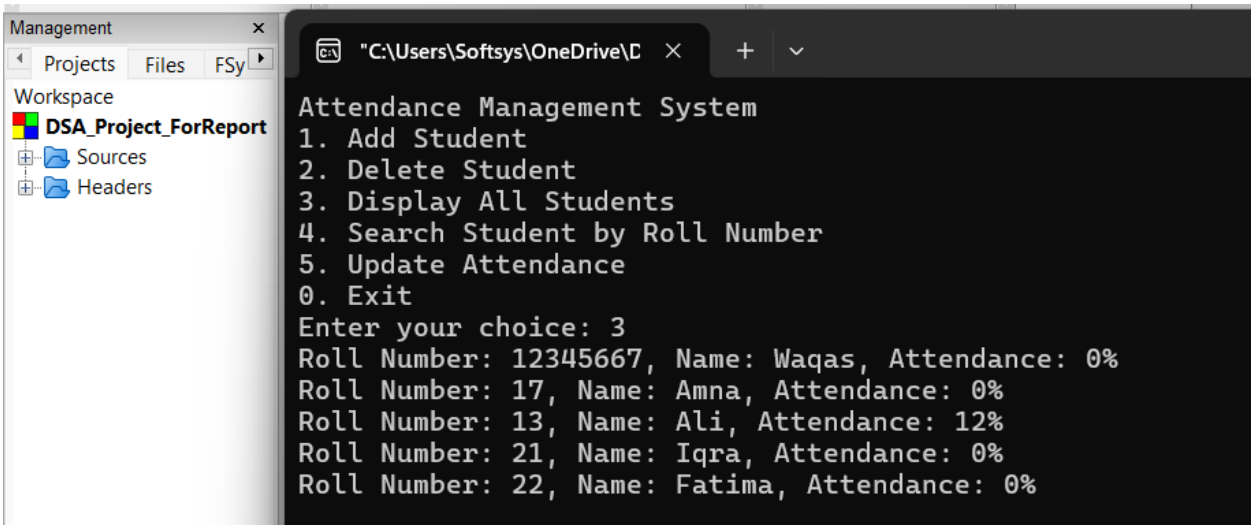
- **Menu Display**: Show the program's menu structure.



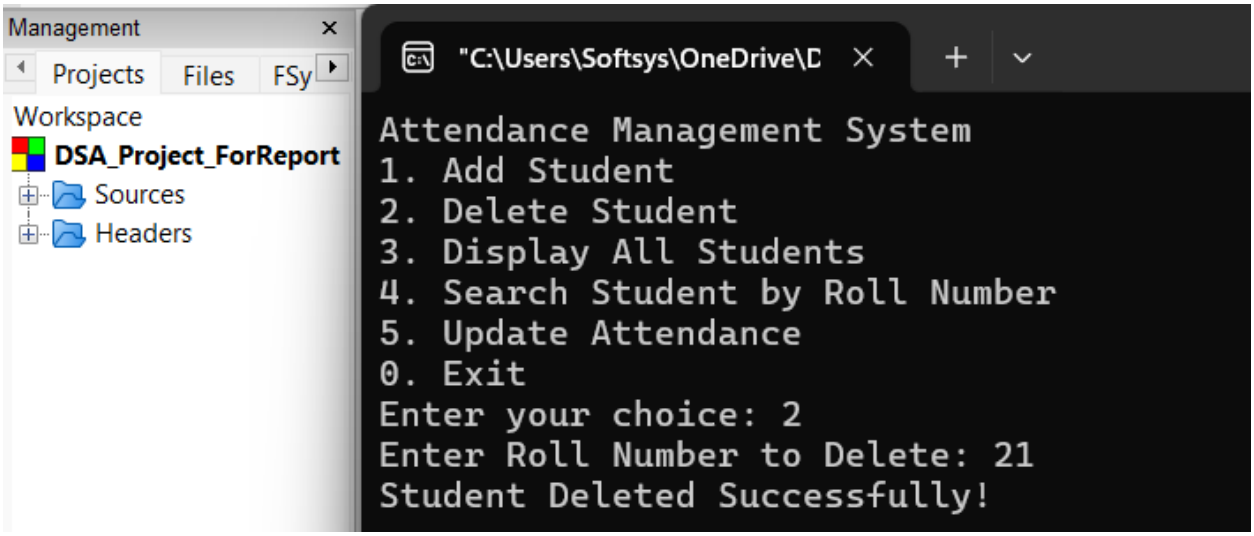
- **Add Student**: Successful and duplicate roll number scenarios.



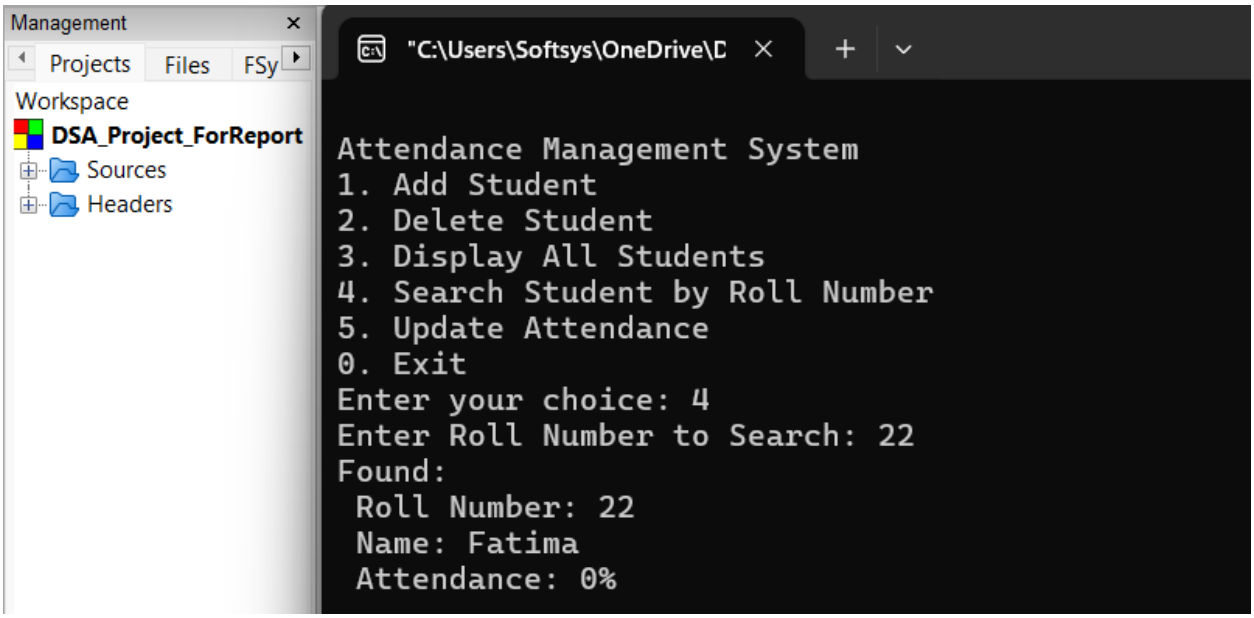
- **Display Records**: Display of all student data.



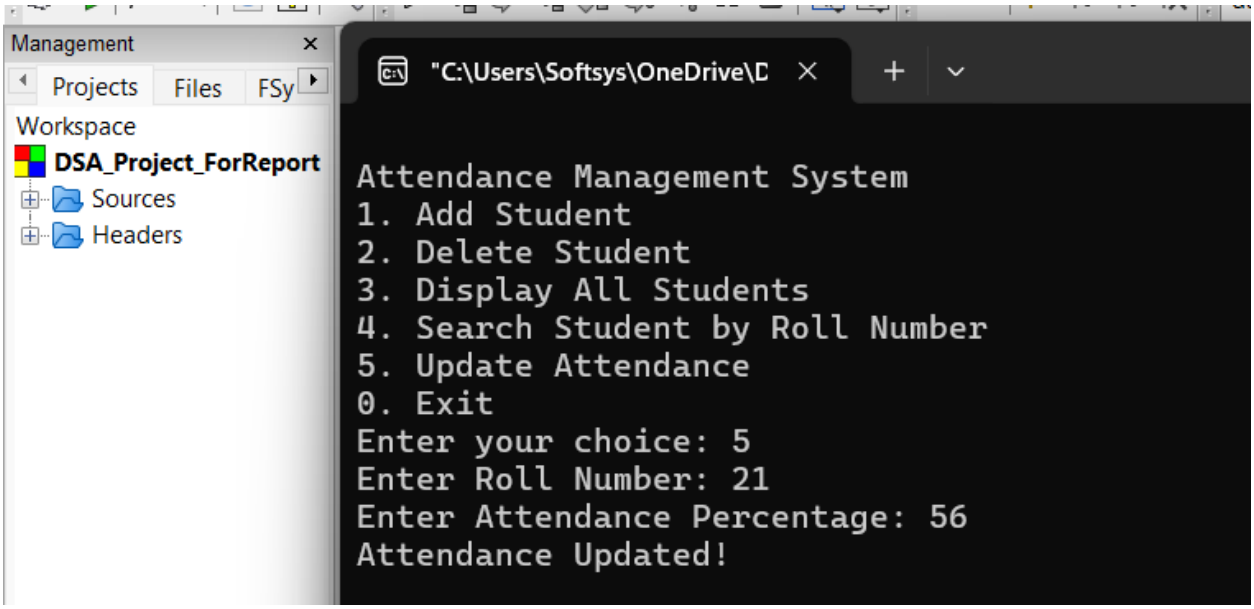
- **Delete Student**: Removal of a record with confirmation.



- **Search Student**: Displaying a single student's data.



- **Update Attendance**: Editing attendance percentage.



Code:

Student Header:

```
#ifndef STUDENT_H

#define STUDENT_H
#include <string>
using namespace std;
//Student class
class Student
 {
public:
     //Student's roll number
     int rollNumber;
    // Student's name
     string name;
    // Student's attendance percentage
     float attendancePercentage;
    // Default constructor
    Student();
    // Parameterized constructor
    Student(int rollNumber,const string& name,float
attendancePercentage = 0.0);
  };
```

```
#endif // STUDENT_H
```

Student cpp:

```
#ifndef STUDENT_H

#include "Student.h"
using namespace std;

// Default constructor
//used in LinkedList and other classes
Student::Student():rollNumber(0),name(""),attendancePercentage(0.0){
}

// Parameterized constructor
// Creating Student object with roll number, name, and attendance
Student::Student(int rollNumber, const string& name,float
attendancePercentage)

:rollNumber(rollNumber),name(name),attendancePercentage(attendancePerc
entage) {
    }
```

LinkedList Header:

```
#ifndef LINKEDLIST_H

#define LINKEDLIST_H
#include "Student.h"
class LinkedList
 {
 public:
      struct Node
      {
        Student student;
        Node* next;
       Node(const Student &student):student(student),next(nullptr)
        {
        }
    };
 private:
    Node* head;
 public:
    LinkedList();  //Constructor
    ~LinkedList();  //Destructor
    Node* GetHead() const;
    void AddStudent(const Student &student);  // Add student
    bool DeleteStudent(int rollNumber);  //Delete student
    void DisplayStudents() const;  //Display
    bool IsEmpty() const;  //Check if list is empty
    bool RollNumberExists(int rollNumber) const;  // Check if this
roll number already exists in the list
};
#endif // LINKEDLIST_H
```

LinkedList Cpp:

```
#include "LinkedList.h"

#include <iostream>
using namespace std;
LinkedList::LinkedList():head(nullptr)
{
}

LinkedList::~LinkedList()
{
    Node* current=head;
    while (current)
        {
        Node* temp=current;
        current=current->next;
        delete temp;
    }
}
void LinkedList::AddStudent(const Student &student)
{
    // Check for duplicate roll number
    Node* temp=head;
    while (temp)
        {
        if (temp->student.rollNumber==student.rollNumber)
        {
            cout << "Error: Roll Number " << student.rollNumber << "
already exists!\n";
            return; //this Prevents duplicate roll number
        }
        temp = temp->next;
    }
    // Add the student to the list
    Node* newNode = new Node(student);
    if (!head) {
        head = newNode;
    } else {
        temp = head;
        while (temp->next) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
bool LinkedList::DeleteStudent(int rollNumber)
{
    Node* temp = head;
    Node* prev = nullptr;
    while (temp)
        {
        if (temp->student.rollNumber == rollNumber)
        {
            if (prev) {
                prev->next = temp->next;
```

```
            } else {
                head = temp->next;
            }
            delete temp;
            return true;
        }
        prev = temp;
        temp = temp->next;
    }
    return false;
}
void LinkedList::DisplayStudents() const
{
    Node* temp = head; //Starts from the head of the list
    if (!temp)
        {
       cout << "No students to display. The list is empty.\n";
        return;
    }
    while (temp)
     {
        cout << "Roll Number: " << temp->student.rollNumber
                  << ", Name: " << temp->student.name
                  << ", Attendance: " << temp-
>student.attendancePercentage << "%" << endl;
        temp = temp->next; //Move to next node
    }
}
bool LinkedList::RollNumberExists(int rollNumber) const
{
    Node* temp = head; // Start from the head
    while (temp)
        {
        if (temp->student.rollNumber == rollNumber)
            {
            return true; //Roll number already exists
        }
        temp = temp->next; //Move to the next node
    }
    return false; //Roll number is unique
}
LinkedList::Node* LinkedList::GetHead() const
 {
    return head; //Return the head of the list
}

bool LinkedList::IsEmpty() const
{
    return head == nullptr; //Return true if the list is empty
}
```

HashTable header:

```
#ifndef HASHTABLE_H

#define HASHTABLE_H
```

```
#include "Student.h"
class HashTable
 {
private:
    struct Node
     {
        Student student;
        Node* next;
        Node(const Student& student) : student(student), next(nullptr)
         {
             //constructor
         }
    };
    static const int tableSize = 100;
    Node* table[tableSize];
    int hashing(int rollNumber) const;
public:
    HashTable();  // Constructor
    ~HashTable();  // Destructor
    void AddStudent(const Student& student);  // Add student to the
hash table
    bool findStudent(int rollNumber, Student& student) const;  // Find
student by roll number
    void updateAttendance(int rollNumber, float attendance);  //
Update attendance for a student
};
#endif
```

HashTable Cpp:

```
#include "HashTable.h"

#include <iostream>
using namespace std;

HashTable::HashTable()
 {
    for(int i=0;i<tableSize;i++)
    {
        table[i]=nullptr;
    }
}
// Destructor
HashTable::~HashTable()
 {
    for(int i=0;i<tableSize;i++)
    {
        Node*current=table[i];
        while(current)
        {
           Node*temp=current;
            current=current->next;
            delete temp;
        }
    }
 }
```

```
//hash index
int HashTable::hashing(int rollNumber)const
 {
    return rollNumber%tableSize;
 }
// Add student
void HashTable::AddStudent(const Student&student)
 {
    int index=hashing(student.rollNumber);

    //duplicate roll number check
    Node*temp=table[index];
    while(temp)
    {
        if(temp->student.rollNumber==student.rollNumber)
         {
            cout<<"Error:Roll Number "<<student.rollNumber<< " already
exists!\n";
            return;
        }
        temp=temp->next;
    }
    // Add the new student
    Node* newNode=new Node(student);
    newNode->next=table[index];
    table[index]=newNode;
}
    //Find student
    bool HashTable::findStudent(int rollNumber,Student& student)const
     {
    int index=hashing(rollNumber);  //Get index
    Node*temp=table[index];
    while(temp)
        {
        if(temp->student.rollNumber==rollNumber)
         {
            student=temp->student;
            return true;
        }
        temp=temp->next;
    }
    return false;
}
    // Update student attendance
    void HashTable::updateAttendance(int rollNumber,float attendance)
{
    int index=hashing(rollNumber);
    Node*temp=table[index];
    while (temp)
        {
        if (temp->student.rollNumber==rollNumber)
        {
            temp->student.attendancePercentage=attendance;  // Update
attendance
            return;
        }
        temp=temp->next;
    }
```

```
    cout<<"Student not found.\n";
}
```

Main:

```
#include <iostream>

#include <string>
#include <limits>
#include <sstream>
#include <fstream>
#include "LinkedList.h"
#include "HashTable.h"
using namespace std;
// Function to validate numeric input
int getValidatedNumber(const string& prompt)
 {
    int num;
    while(true)
    {
        cout<<prompt;
        cin>>num;
        if(cin.fail())
     {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(),'\n');
            cout<<"Invalid input! Please enter a number.\n";
    }
     else
        {
            cin.ignore(numeric_limits<streamsize>::max(),'\n');
            return num;
        }
    }
}
string getValidatedName()
{
    string name;
    while(true)
     {
        cout <<"Enter Name: ";
        getline(cin,name);
        bool isValid = true;
        for (char c : name) {
            if (!isalpha(c) && c!=' ')
       {
                isValid = false;
                break;
         }
        }
        if(isValid&&!name.empty())
        {
            return name; // Return the valid name
        } else
         {
            cout<< "Invalid name! Please enter a valid alphabetic
name.\n";
        }
    }
```

```
}
// Save student data
void saveToFile(const LinkedList& studentList,const string& filename)
 {
    ofstream outFile(filename);
    if (!outFile)
        {
        cerr<<"Error opening file for writing!\n";
        return;
    }
    LinkedList::Node*temp=studentList.GetHead();  // Get head from
LinkedList
    while(temp)
     {
        outFile<<temp->student.rollNumber<<","
               <<temp->student.name<<","
               << temp->student.attendancePercentage << "\n";
        temp = temp->next;
    }
    outFile.close();
}

// Load student data from a file
void loadFromFile(LinkedList& studentList,HashTable&
studentTable,const std::string& filename)
 {
    ifstream inFile(filename);
    if(!inFile)
        {
        cerr<<"Starting with an empty database.\n";
        return;
    }
     string line;
    while(getline(inFile,line))
    {
        stringstream ss(line);
        int rollNumber;
        string name;
        float attendance;
        char comma;
        ss >> rollNumber>> comma;
        getline(ss, name,',');
        ss >> attendance;
        Student student(rollNumber, name, attendance);
        studentList.AddStudent(student);
        studentTable.AddStudent(student);
    }
    inFile.close();
}
int main()
 {
    int choice;
    LinkedList studentList;
    HashTable studentTable;
    const string filename="students.csv";

    // Load data from file at the start
    loadFromFile(studentList, studentTable, filename);
```

```
    do {
        cout<<"\nAttendance Management System\n";
        cout<<"1. Add Student\n2. Delete Student\n3. Display All
Students\n";
        cout<<"4. Search Student by Roll Number\n5. Update
Attendance\n0. Exit\n";
        choice=getValidatedNumber("Enter your choice: ");
    switch (choice)
     {
       case 1:
       {
        string name=getValidatedName();  // ensure valid name input
        int rollNumber=getValidatedNumber("Enter Roll Number: ");
        if(studentList.RollNumberExists(rollNumber))
        {
        cout<<"Error: Roll Number "<< rollNumber << " is already
assigned to another student.\n";
        break;
    }
    Student student(rollNumber, name);
    studentList.AddStudent(student);
    studentTable.AddStudent(student);
    saveToFile(studentList,filename); // Save data after addition
    break;
}
    case 2:
        {
            int rollNumber=getValidatedNumber("Enter Roll Number to
Delete: ");
            if (studentList.DeleteStudent(rollNumber))
        {
                saveToFile(studentList, filename); // Save data after
deletion
             cout<<"Student Deleted Successfully!\n";
         }
        else
          {
             cout<<"Student Not Found!\n";
              }
                 break;
              }
     case 3:
       if (studentList.GetHead()==nullptr)
        {
             cout<<"Student list is empty!\n";
         }
        else
          {
            studentList.DisplayStudents();
            }
                break;
        case 4:
          {
            int rollNumber=getValidatedNumber("Enter Roll Number to
Search: ");
             Student student(0,"");
             if (studentTable.findStudent(rollNumber, student))
```

```
                {
                  cout<<"Found:"<<"\n"<<" Roll Number:
"<<student.rollNumber<<"\n"<<" Name: "<<student.name<<"\n"<<"
Attendance: "<<student.attendancePercentage<<"%\n";
                }
                else
                 {
                    cout<<"Student Not Found!\n";
                }
                break;
            }
        case 5:
            {
               int rollNumber=getValidatedNumber("Enter Roll Number:
");
                float attendance;
                cout<<"Enter Attendance Percentage: ";
                cin>>attendance;
                cin.ignore(numeric_limits<streamsize>::max(),'\n');
                Student student(0, "");
                if (studentTable.findStudent(rollNumber, student))
                    {
                    student.attendancePercentage = attendance;
                    studentTable.updateAttendance(rollNumber,
attendance);
                    studentList.DeleteStudent(rollNumber);
                    studentList.AddStudent(student);
                    saveToFile(studentList, filename); // Save data
after update
                   cout<<"Attendance Updated!\n";
                }
                else
                    {
                    cout<<"Student Not Found!\n";
                }
                break;
            }
            case 0:
                cout<<"Exiting...\n";
                break;

            default:
                cout<<"Invalid Choice! Try Again.\n";
        }
    }
    while (choice != 0);
    return 0;
}
```